```
Array
(
    [0] => Alice
    [1] => Russell
)
Array
(
    [0] => Bob
    [1] => Russell
)
```

When we remove an element from an array (in this case, using the `unset()` function), the indexes are not automatically updated, as we can see in the output of the first `print_r()`. After unsetting `$myArr[1]`, the second element in the array is now `"Russell"`, but the index for that element is still `2`. The index sequence is now `0`, `2`.

Two built-in functions, `array_splice()` and `array_values()`, are used here to force reindexing of arrays after an element is removed. In the case of `array_values()`, the elements of the input array are returned but with a new index; we then overwrite `$myArr1` with that returned array. We've already seen `array_splice()` in action; you might have noticed then that when we inserted new elements, the index values of all elements were updated. The outcome for each function in this example is identical; the only difference between using the two functions in this way is that `array_values()` will outperform `array_splice()` by about 10%.

Splitting Arrays

If for any reason you need to divide an array into equally sized sets, the built-in function `array_chunk()` will allow you to do just that:

```
$arr = [
    'a' => 'Alice',
    'b' => 'Bob',
    'r' => 'Russell',
];
```